

malloc(3) — Linux manual page

[NAME](#) | [LIBRARY](#) | [SYNOPSIS](#) | [DESCRIPTION](#) | [RETURN VALUE](#) | [ERRORS](#) | [ATTRIBUTES](#) | [STANDARDS](#) | [HISTORY](#) | [NOTES](#) | [EXAMPLES](#) | [SEE ALSO](#) | [COLOPHON](#)

 malloc(3)

Library Functions Manual

malloc(3)**NAME** [top](#)

malloc, free, calloc, realloc, reallocarray - allocate and free dynamic memory

LIBRARY [top](#)

Standard C library (*libc*, *-lc*)

SYNOPSIS [top](#)

```
#include <stdlib.h>
```

```
void *malloc(size_t size);
void free(void *_Nullable ptr);
void *calloc(size_t n, size_t size);
void *realloc(void *_Nullable ptr, size_t size);
void *reallocarray(void *_Nullable ptr, size_t n, size_t size);
```

Feature Test Macro Requirements for glibc (see [feature_test_macros\(7\)](#)):

```
reallocarray():
    Since glibc 2.29:
        _DEFAULT_SOURCE
    glibc 2.28 and earlier:
        _GNU_SOURCE
```

DESCRIPTION [top](#)**malloc()**

The **malloc()** function allocates *size* bytes and returns a pointer to the allocated memory. *The memory is not initialized.* If *size* is 0, then **malloc()** returns a unique pointer value that can later be successfully passed to **free()**. (See "Nonportable behavior" for portability issues.)

free()

The **free()** function frees the memory space pointed to by *ptr*, which must have been returned by a previous call to **malloc()** or related functions. Otherwise, or if *ptr* has already been freed, undefined behavior occurs. If *ptr* is NULL, no operation is performed.

calloc()

The **calloc()** function allocates memory for an array of *n* elements of *size* bytes each and returns a pointer to the allocated memory.

The memory is set to zero. If *n* or *size* is 0, then **calloc()** returns a unique pointer value that can later be successfully passed to **free()**.

If the multiplication of *n* and *size* would result in integer overflow, then **calloc()** returns an error. By contrast, an integer overflow would not be detected in the following call to **malloc()**, with the result that an incorrectly sized block of memory would be allocated:

```
malloc(n * size);
```

realloc()

The **realloc()** function changes the size of the memory block pointed to by *ptr* to *size* bytes. The contents of the memory will be unchanged in the range from the start of the region up to the minimum of the old and new sizes. If the new size is larger than the old size, the added memory will *not* be initialized.

If *ptr* is NULL, then the call is equivalent to **malloc(size)**, for all values of *size*.

If *size* is equal to zero, and *ptr* is not NULL, then the call is equivalent to **free(ptr)** (but see "Nonportable behavior" for portability issues).

Unless *ptr* is NULL, it must have been returned by an earlier call to **malloc** or related functions. If the area pointed to was moved, a **free(ptr)** is done.

reallocarray()

The **reallocarray()** function changes the size of (and possibly moves) the memory block pointed to by *ptr* to be large enough for an array of *n* elements, each of which is *size* bytes. It is equivalent to the call

```
realloc(ptr, n * size);
```

However, unlike that **realloc()** call, **reallocarray()** fails safely in the case where the multiplication would overflow. If such an overflow occurs, **reallocarray()** returns an error.

RETURN VALUE [top](#)

The **malloc()**, **calloc()**, **realloc()**, and **reallocarray()** functions return a pointer to the allocated memory, which is suitably aligned for any type that fits into the requested size or less. On error, these functions return NULL and set *errno*. Attempting to allocate more than **PTRDIFF_MAX** bytes is considered an error, as an object that large could cause later pointer subtraction to overflow.

The **free()** function returns no value, and preserves *errno*.

The **realloc()** and **reallocarray()** functions return NULL if *ptr* is not NULL and the requested size is zero; this is not considered an error. (See "Nonportable behavior" for portability issues.) Otherwise, the returned pointer may be the same as *ptr* if the allocation was not moved (e.g., there was room to expand the allocation in-place), or different from *ptr* if the allocation was moved to a new address. If these functions fail, the original block is left untouched; it is not freed or moved.

ERRORS [top](#)

calloc(), **malloc()**, **realloc()**, and **reallocarray()** can fail with the following error:

ENOMEM Out of memory. Possibly, the application hit the **RLIMIT_AS** or **RLIMIT_DATA** limit described in [getrlimit\(2\)](#). Another reason could be that the number of mappings created by the caller process exceeded the limit specified by [/proc/sys/vm/max_map_count](#).

ATTRIBUTES [top](#)

For an explanation of the terms used in this section, see [attributes\(7\)](#).

Interface	Attribute	Value
malloc() , free() , calloc() , realloc()	Thread safety	MT-Safe

STANDARDS [top](#)

malloc()
free()
calloc()
realloc()
C11, POSIX.1-2008.

reallocarray()
None.

HISTORY [top](#)

malloc()
free()
calloc()
realloc()
POSIX.1-2001, C89.

reallocarray()
glibc 2.26. OpenBSD 5.6, FreeBSD 11.0.

malloc() and related functions rejected sizes greater than **PTRDIFF_MAX** starting in glibc 2.30.

free() preserved [errno](#) starting in glibc 2.33.

NOTES [top](#)

By default, Linux follows an optimistic memory allocation strategy. This means that when **malloc()** returns non-NULL there is no guarantee that the memory really is available. In case it turns out that the system is out of memory, one or more processes will be killed by the OOM killer. For more information, see the description of [/proc/sys/vm/overcommit_memory](#) and [/proc/sys/vm/oom_adj](#) in [proc\(5\)](#), and the Linux kernel source file [Documentation/vm/overcommit-accounting.rst](#).

Normally, **malloc()** allocates memory from the heap, and adjusts the size of the heap as required, using **sbrk(2)**. When allocating blocks of memory larger than **MMAP_THRESHOLD** bytes, the glibc **malloc()** implementation allocates the memory as a private anonymous mapping using **mmap(2)**. **MMAP_THRESHOLD** is 128 kB by default, but is adjustable using **mallopt(3)**. Prior to Linux 4.7 allocations performed using **mmap(2)** were unaffected by the **RLIMIT_DATA** resource limit; since Linux 4.7, this limit is also enforced for allocations performed using **mmap(2)**.

To avoid corruption in multithreaded applications, mutexes are used internally to protect the memory-management data structures employed by these functions. In a multithreaded application in which threads simultaneously allocate and free memory, there could be contention for these mutexes. To scalably handle memory allocation in multithreaded applications, glibc creates additional *memory allocation arenas* if mutex contention is detected. Each arena is a large region of memory that is internally allocated by the system (using **brk(2)** or **mmap(2)**), and managed with its own mutexes.

If your program uses a private memory allocator, it should do so by replacing **malloc()**, **free()**, **calloc()**, and **realloc()**. The replacement functions must implement the documented glibc behaviors, including **errno** handling, size-zero allocations, and overflow checking; otherwise, other library routines may crash or operate incorrectly. For example, if the replacement **free()** does not preserve **errno**, then seemingly unrelated library routines may fail without having a valid reason in **errno**. Private memory allocators may also need to replace other glibc functions; see "Replacing malloc" in the glibc manual for details.

Crashes in memory allocators are almost always related to heap corruption, such as overflowing an allocated chunk or freeing the same pointer twice.

The **malloc()** implementation is tunable via environment variables; see **mallopt(3)** for details.

Nonportable behavior

The behavior of these functions when the requested size is zero is glibc specific; other implementations may return NULL without setting **errno**, and portable POSIX programs should tolerate such behavior. See **realloc(3p)**.

POSIX requires memory allocators to set **errno** upon failure. However, the C standard does not require this, and applications portable to non-POSIX platforms should not assume this.

Portable programs should not use private memory allocators, as POSIX and the C standard do not allow replacement of **malloc()**, **free()**, **calloc()**, and **realloc()**.

EXAMPLES [top](#)

```
#include <err.h>
#include <stddef.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MALLOCARRAY(n, type) ((type *) my_mallocarray(n, sizeof(type)))
#define MALLOC(type) MALLOCARRAY(1, type)
```

```
static inline void *my_mallocarray(size_t n, size_t size);

int
main(void)
{
    char *p;

    p = MALLOCARRAY(32, char);
    if (p == NULL)
        err(EXIT_FAILURE, "malloc");

    strcpy(p, "foo", 32);
    puts(p);
}

static inline void *
my_mallocarray(size_t n, size_t size)
{
    return reallocarray(NULL, n, size);
}
```

SEE ALSO [top](#)

[valgrind\(1\)](#), [brk\(2\)](#), [mmap\(2\)](#), [alloca\(3\)](#), [malloc_get_state\(3\)](#), [malloc_info\(3\)](#), [malloc_trim\(3\)](#), [malloc_usable_size\(3\)](#), [mallopt\(3\)](#), [mcheck\(3\)](#), [mtrace\(3\)](#), [posix_memalign\(3\)](#)

For details of the GNU C library implementation, see <https://sourceware.org/glibc/wiki/MallocInternals>.

COLOPHON [top](#)

This page is part of the *man-pages* (Linux kernel and C library user-space interface documentation) project. Information about the project can be found at <https://www.kernel.org/doc/man-pages/>. If you have a bug report for this manual page, see <https://git.kernel.org/pub/scm/docs/man-pages/man-pages.git/tree/CONTRIBUTING>. This page was obtained from the tarball `man-pages-6.10.tar.gz` fetched from <https://mirrors.edge.kernel.org/pub/linux/docs/man-pages/> on 2025-02-02. If you discover any rendering problems in this HTML version of the page, or you believe there is a better or more up-to-date source for the page, or you have corrections or improvements to the information in this COLOPHON (which is *not* part of the original manual page), send a mail to `man-pages@man7.org`

Linux man-pages 6.10**2024-11-17*****malloc(3)***

Pages that refer to this page: [memusage\(1\)](#), [brk\(2\)](#), [clone\(2\)](#), [getrlimit\(2\)](#), [io_uring_register\(2\)](#), [mlock\(2\)](#), [mmap\(2\)](#), [mremap\(2\)](#), [alloca\(3\)](#), [argz_add\(3\)](#), [asprintf\(3\)](#), [ausearch_add_expression\(3\)](#), [avc_init\(3\)](#), [backtrace\(3\)](#), [basename\(3\)](#), [canonicalize_file_name\(3\)](#), [cfree\(3\)](#), [CPU_SET\(3\)](#), [dbopen\(3\)](#), [end\(3\)](#), [exec\(3\)](#), [fopen\(3\)](#), [fseek\(3\)](#), [fts\(3\)](#), [ftw\(3\)](#), [getcwd\(3\)](#), [getgrent\(3\)](#), [getgrnam\(3\)](#), [getifaddrs\(3\)](#), [getline\(3\)](#), [getpwent\(3\)](#), [getpwnam\(3\)](#), [glob\(3\)](#), [hsearch\(3\)](#), [if_nameindex\(3\)](#), [lber-memory\(3\)](#), [ldap_memory\(3\)](#), [mallinfo\(3\)](#), [malloc_get_state\(3\)](#), [malloc_hook\(3\)](#), [malloc_info\(3\)](#), [malloc_stats\(3\)](#), [malloc_trim\(3\)](#), [malloc_usable_size\(3\)](#), [mallopt\(3\)](#), [mcheck\(3\)](#), [mpool\(3\)](#), [mtrace\(3\)](#), [numa\(3\)](#), [open_memstream\(3\)](#), [pam_conv\(3\)](#), [pmaddderived\(3\)](#), [__pmaf\(3\)](#), [pmagetlog\(3\)](#), [pmapi\(3\)](#), [pmarewritedata\(3\)](#), [pmarewritemeta\(3\)](#), [pmdachildren\(3\)](#), [pmdafetch\(3\)](#), [pmdainstance\(3\)](#), [pmdalabel\(3\)](#), [pmdatext\(3\)](#), [pmdatrace\(3\)](#), [pmdiscoverservices\(3\)](#), [pmextractvalue\(3\)](#), [pmfault\(3\)](#), [pmfetch\(3\)](#), [pmfetcharchive\(3\)](#), [pmfetchgroup\(3\)](#), [pmfreelabelsets\(3\)](#), [pmfreeprofile\(3\)](#), [pmfreeresult\(3\)](#), [pmfstring\(3\)](#), [pmgetchildren\(3\)](#), [pmgetchildrenstatus\(3\)](#), [pmgetindom\(3\)](#), [pmgetindomarchive\(3\)](#), [pmhash\(3\)](#), [pmlookupindomtext\(3\)](#), [pmlookuptext\(3\)](#), [pmnameall\(3\)](#),

[pmnameid\(3\)](#), [pmnameindom\(3\)](#), [pmnameindomarchive\(3\)](#), [pmnewcontextzone\(3\)](#), [pmnewzone\(3\)](#), [pmnomem\(3\)](#), [__pmparsectime\(3\)](#), [pmparsehostattrspec\(3\)](#), [pmparsehostspec\(3\)](#), [pmparseinterval\(3\)](#), [pmparsemetricspec\(3\)](#), [__pmparsetime\(3\)](#), [pmparsetimewindow\(3\)](#), [pmparseunitsstr\(3\)](#), [pmregisterderived\(3\)](#), [posix_memalign\(3\)](#), [pthread_setcancelstate\(3\)](#), [random_r\(3\)](#), [readdir\(3\)](#), [readline\(3\)](#), [realpath\(3\)](#), [scandir\(3\)](#), [sd_bus_creds_get_pid\(3\)](#), [sd_bus_error\(3\)](#), [sd_bus_path_encode\(3\)](#), [sd_get_seats\(3\)](#), [sd_journal_get_catalog\(3\)](#), [sd_journal_get_cursor\(3\)](#), [sd-json\(3\)](#), [sd-login\(3\)](#), [sd_machine_get_class\(3\)](#), [sd_path_lookup\(3\)](#), [sd_pid_get_owner_uid\(3\)](#), [sd_seat_get_active\(3\)](#), [sd_session_is_active\(3\)](#), [sd_uid_get_state\(3\)](#), [seccomp_syscall_resolve_name\(3\)](#), [security_class_to_string\(3\)](#), [selabel_get_digests_all_partial_matches\(3\)](#), [selinux_boolean_sub\(3\)](#), [selinux_getpolicytype\(3\)](#), [selinux_raw_context_to_color\(3\)](#), [setbuf\(3\)](#), [sscanf\(3\)](#), [strdup\(3\)](#), [string\(3\)](#), [tempnam\(3\)](#), [tracefs_event_get_file\(3\)](#), [tracefs_instance_set_affinity\(3\)](#), [tracefs_tracers\(3\)](#), [void\(3type\)](#), [wcsdup\(3\)](#), [proc_meminfo\(5\)](#), [environ\(7\)](#), [feature_test_macros\(7\)](#), [signal-safety\(7\)](#), [string_copying\(7\)](#)

HTML rendering created 2025-02-02 by [Michael Kerrisk](#), author of *The Linux Programming Interface*.

For details of in-depth **Linux/UNIX system programming training courses** that I teach, look [here](#).

Hosting by [jambit GmbH](#).

